

Wireless 0-10 Volt Dimmer

Generated by Doxygen 1.9.8

1 Wireless 0-10 volt light dimmer	1
1.1 Design and implementation notes	1
1.1.1 Goals	1
1.1.2 Approach	1
1.1.3 Firmware notes	2
1.1.3.1 Configuration	2
1.1.3.2 Threads	2
1.1.3.3 Shared state	2
1.1.4 Rx driver circuit	3
1.1.4.1 All purpose option	3
1.1.4.2 Pulled inputs option	3
2 Class Index	5
2.1 Class List	5
3 File Index	7
3.1 File List	7
4 Class Documentation	9
4.1 checkpoint Struct Reference	9
4.1.1 Detailed Description	9
4.2 dimmer_payload Struct Reference	9
4.2.1 Detailed Description	10
4.3 led Struct Reference	10
4.3.1 Detailed Description	10
4.3.2 Member Data Documentation	10
4.3.2.1 led_state	10
4.4 level_encoder Struct Reference	11
4.4.1 Detailed Description	12
4.5 pwm Struct Reference	12
4.5.1 Detailed Description	12
4.6 shared Struct Reference	12
4.6.1 Detailed Description	13
4.7 strap Struct Reference	13
4.7.1 Detailed Description	13
4.8 wifi_payload Struct Reference	14
4.8.1 Detailed Description	14
5 File Documentation	15
5.1 dimmer.c File Reference	15
5.1.1 Detailed Description	16
5.2 encoder.c File Reference	17
5.2.1 Detailed Description	17
5.2.2 Function Documentation	18

5.2.2.1 initialize_level_encoder()	18
5.3 encoder.h File Reference	18
5.3.1 Detailed Description	19
5.3.2 Function Documentation	19
5.3.2.1 initialize_level_encoder()	19
5.4 encoder.h	20
5.5 led.h File Reference	20
5.5.1 Detailed Description	22
5.5.2 Function Documentation	22
5.5.2.1 initialize_led()	22
5.5.2.2 set_led_flash_count()	22
5.6 led.h	22
5.7 pwm.h File Reference	23
5.7.1 Detailed Description	24
5.7.2 Function Documentation	24
5.7.2.1 initialize_pwm()	24
5.7.2.2 set_pwm_duty_mils()	24
5.8 pwm.h	25
5.9 shared.h File Reference	25
5.9.1 Detailed Description	27
5.10 shared.h	27
5.11 strap.h File Reference	27
5.11.1 Detailed Description	28
5.11.2 Function Documentation	29
5.11.2.1 initialize_strap()	29
5.12 strap.h	29
5.13 wifi.h File Reference	29
5.13.1 Detailed Description	30
5.13.2 Function Documentation	30
5.13.2.1 initialize_wifi()	30
5.13.2.2 send_repeated()	30
5.14 wifi.h	31
5.15 led.c File Reference	31
5.15.1 Detailed Description	32
5.15.2 Function Documentation	32
5.15.2.1 initialize_led()	32
5.15.2.2 set_led_flash_count()	32
5.16 pwm.c File Reference	33
5.16.1 Detailed Description	34
5.16.2 Function Documentation	34
5.16.2.1 initialize_pwm()	34
5.16.2.2 set_pwm_duty_mils()	34

5.17 shared.c File Reference	34
5.17.1 Detailed Description	35
5.18 strap.c File Reference	35
5.18.1 Detailed Description	36
5.18.2 Function Documentation	36
5.18.2.1 initialize_strap()	36
5.19 wifi.c File Reference	37
5.19.1 Detailed Description	38
5.19.2 Function Documentation	38
5.19.2.1 initialize_wifi()	38
5.19.2.2 send_repeated()	38
Index	39

Chapter 1

Wireless 0-10 volt light dimmer

1.1 Design and implementation notes

This design is for the case of 0-10 volt dimmable fixtures controlled by multiple switches replacing old fixtures that either weren't dimmable or used mains dimming, e.g. standard triac-based devices. Therefore either new 0-10 volt wiring or a wireless solution is needed. This project is about the second approach.

It is my first ESP32 project. All feedback is very welcome. Be kind.

1.1.1 Goals

- Retain existing on/off switches. No mains power switching by the dimmer.
 - Avoids safety considerations and complexity.
- Receiver (rx) box at each light running on switched light power, producing the 0-10 volt control.
- Transmitter (tx) box with a knob at each switch (or anywhere else). Twisting any changes the 0-10 volt output of all receivers.
- No batteries. Mains power for all boxes.
- Secure: Reasonably proof against playback and spoofing attacks.
- Power interruption resilience: doesn't lose dimming level and keeps working.

1.1.2 Approach

- ESP32-WROOM dev board for each tx and rx.
- ESP-IDF environment because several threads will be needed.
 - Looks more full-featured than Arduino.
- ESP-NOW wifi broadcast protocol for communication.
 - Repeated broadcasts for robustness. No tx/rx handshake.
- Messages secured via checkpointed sequence numbers and 128-bit hmac.
 - Hard-coded hmac password to avoid distribution complexity.

- Rotary quadrature encoder for tx knob. Polled for simplicity.
- Checkpoints include sequence current dimming level for power off resilience.
 - ESP32 non-volatile storage (nvs) written with de-bouncing to conserve flash life.
- Receiver PWM output to simple op-amp based 0-10 driver.
- On-board LED as state indicator.
- 3d-printed cases.
 - Tx case integrated with light switch plate.
 - Rx is a "cord bump" with mains plug entering one end and 0-10 volt control line the other.

1.1.3 Firmware notes

Single firmware load for tx and rx. Ground strap on pin RECV_STRAP_GPIO in `dimmer.c` makes the unit an rx. Else it's a tx.

1.1.3.1 Configuration

The default ESP-IDF project configuration had to be modified.

- FreeRTOS timer tick rate (`CONFIG_FREERTOS_HZ`) to 1000
 - Supports rotary encoder polling.
- Flash size to 4Mb. (`CONFIG_ESPTOOLPY_FLASHSIZE_4MB` and `CONFIG_ESPTOOLPY_FLASHSIZE`) Boot logging suggested this.

Other dev boards may require changing `ONBOARD_LED_GPIO` in `led.h` and other GPIO assignments. All other GPIO assignments are in `dimmer.c` and may be reconfigured at will subject to the normal caveats.

1.1.3.2 Threads

- Wifi stack making standard callbacks.
- Broadcast repeats. Async after initial trigger. Cancelled by new trigger.
- Flashing LED via FreeRTOS timer state machine.
- Rotary encoder via FreeRTOS timer-based polling.
- Checkpoints to nvs, debounced to conserve nvs writes.

1.1.3.3 Shared state

The module `shared.c/h` houses data touched by several threads, therefore implemented with `<stdatomic.h>`. These are exactly sufficient for power fail resilience checkpoints.

- Broadcast sequence number
- Current dimmer level

There is another small block of data protected by mutex in `wifi.c`. It contains state for sending broadcasts repeatedly. The triggering thread sets up this data while holding the mutex, then wakes up the repeat thread, which iterates the requested number of times, grabbing the mutex to ensure no races with triggering threads.

1.1.4 Rx driver circuit

There are two options: one that will always work and another depends on a particular implementation of the driven lights.

Both options start with a 2-pole RC low pass filter to convert the 3.3 volt PWM output to a 0-3.3 volt DC signal. "Soft start" is also desired: forcing a 0v output for about one second while the receiver boots and sets up the PWM output at turn-on time.

1.1.4.1 All purpose option

Use a standard 3.03x non-inverting op amp circuit (LM385 or similar) to convert 0-3.3 volts to 0-10. The op amp requires a 12 volt supply in this case, while the ESP32 needs either 3.3 or 5 volts. A dual voltage supply is necessary.

1.1.4.2 Pulled inputs option

If the controlled lights have a 10v (or more) default potential across the control line, then the op amp can drive a BJT that serves only to draw down this potential to the dimming level. A 1:3 voltage divider in the feedback loop lets the op amp bias the transistor properly using only a 5-volt supply. This is handy, as it allows receivers to work with a single rather than dual voltage supply.

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

checkpoint	Persistent checkpoint	9
dimmer_payload	Packed broadcast data payload	9
led	Data for a timer-driven flashing LED	10
level_encoder	State of a level encoder	11
pwm	Data for a GPIO pin serving as a PWM output	12
shared	State shared among threads and/or checkpointed to persistent store	12
strap	Data for a strap	13
wifi_payload	Packed wifi payload wrapping the user's data	14

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

dimmer.c	Wireless 0-10 volt dimmer app main	15
encoder.c	Rotary level encoder implementation	17
encoder.h	Rotary level encoder	18
led.h	Variable count flashing LED	20
pwm.h	PWM output	23
shared.h	Multi-thread shared and/or checkpointed state	25
strap.h	Configuration strap	27
wifi.h	Wifi broadcast and receive singleton	29
led.c	Variable count flashing LED implementation	31
pwm.c	PWM output implementation	33
shared.c	Multi-thread shared and/or checkpointed state implementation	34
strap.c	Configuration strap implementation	35
wifi.c	Wifi broadcast and receive singleton implementation	37

Chapter 4

Class Documentation

4.1 checkpoint Struct Reference

Persistent checkpoint.

```
#include <shared.h>
```

Public Attributes

- `uint32_t sequence`
Broadcast sequence number.
- `uint16_t level_mils`
Dimming level in [0..1024].

4.1.1 Detailed Description

Persistent checkpoint.

The documentation for this struct was generated from the following file:

- [shared.h](#)

4.2 dimmer_payload Struct Reference

Packed broadcast data payload.

Public Attributes

- `uint16_t level_mils`
Dimming level in [0..1024].

4.2.1 Detailed Description

Packed broadcast data payload.

The documentation for this struct was generated from the following file:

- [dimmer.c](#)

4.3 led Struct Reference

Data for a timer-driven flashing LED.

```
#include <led.h>
```

Public Attributes

- char * **name**
Name of the LED for logging.
- gpio_num_t **gpio**
GPIO pin driving the LED.
- StaticTimer_t **timer_state** [1]
Blink timer state.
- TimerHandle_t **timer**
Blink timer handle.
- uint16_t **led_state**
Index connoting which flashing phase the led is in.
- uint8_t **last_led_state**
Two times desired_flash_count.

4.3.1 Detailed Description

Data for a timer-driven flashing LED.

4.3.2 Member Data Documentation

4.3.2.1 led_state

```
uint16_t led::led_state
```

Index connoting which flashing phase the led is in.

States:

- 0 - long off
- 1 - first on
- 2 - first short off
- ...
- last_led_state-1 - last on (followed by 0 = long off)

The documentation for this struct was generated from the following file:

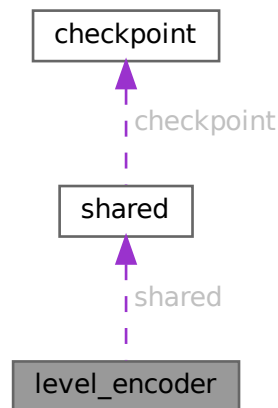
- [led.h](#)

4.4 level_encoder Struct Reference

State of a level encoder.

```
#include <encoder.h>
```

Collaboration diagram for level_encoder:



Public Attributes

- struct `shared` * **shared**
Shared state variables. Level lives here.
- char * **name**
Encoder name for logging.
- uint8_t **sw_gpio**
GPIO input pin: push switch.
- uint8_t **clk_gpio**
GPIO input pin: quadrature A.
- uint8_t **dt_gpio**
GPIO input pin: quadrature B.
- uint16_t **level_max**
Number of quadrature transitions to consider "full on".
- uint8_t **sw_value**
Push-button switch state. 0=presented, 1=not.
- uint16_t **level**
Last valid encoded level in quadrature ticks. Internal.
- void(* **on_level_change**)(struct `level_encoder` *)
Encoder position change callback.
- void(* **on_sw_change**)(struct `level_encoder` *)
Pushbutton press/release callback.
- StaticTimer_t **timer_state** [1]
Blink timer state. Internal.
- TimerHandle_t **timer**
Blink timer. Internal.
- int32_t **last_state**
Last encoder quadrature state. 2 bits.

4.4.1 Detailed Description

State of a level encoder.

The documentation for this struct was generated from the following file:

- [encoder.h](#)

4.5 pwm Struct Reference

Data for a GPIO pin serving as a PWM output.

```
#include <pwm.h>
```

Public Attributes

- char * **name**
*Name of the PWM output for logging. */.*
- gpio_num_t **gpio**
*GPIO pin for PWM output. */.*
- uint16_t **duty**
*Raw internal duty cycle. */.*

4.5.1 Detailed Description

Data for a GPIO pin serving as a PWM output.

The documentation for this struct was generated from the following file:

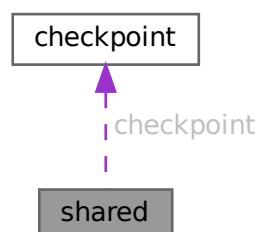
- [pwm.h](#)

4.6 shared Struct Reference

State shared among threads and/or checkpointed to persistent store.

```
#include <shared.h>
```

Collaboration diagram for shared:



Public Attributes

- `_Atomic uint32_t sequence`
Broadcast sequence number.
- `_Atomic uint16_t level_mils`
Dimming level in [0..1024].
- struct `checkpoint checkpoint` [1]
The last data saved as a checkpoint.
- `nvs_handle_t nvs`
Non-volatile storage.
- `TaskHandle_t checkpoint_task`
Checkpoint manager task.
- `StaticTask_t checkpoint_task_state` [1]
Checkpoint manager task state.
- `StackType_t checkpoint_task_stack` [`CHECKPOINT_TASK_STACK_SIZE`]
Checkpoint manager stack.

4.6.1 Detailed Description

State shared among threads and/or checkpointed to persistent store.

The documentation for this struct was generated from the following file:

- [shared.h](#)

4.7 strap Struct Reference

Data for a strap.

```
#include <strap.h>
```

Public Attributes

- `char * name`
Name of the strap for logging.
- `uint8_t value`
Value of the strap at initialization time.

4.7.1 Detailed Description

Data for a strap.

The documentation for this struct was generated from the following file:

- [strap.h](#)

4.8 wifi_payload Struct Reference

Packed wifi payload wrapping the user's data.

Public Attributes

- uint32_t **sequence**
Broadcast sequence number.
- uint8_t **hmac** [16]
Secure hash based on payload contents.
- uint16_t **data_len**
Actual user data size (bytes).
- uint8_t **data** [[MAX_PAYLOAD_DATA_SIZE](#)]
User data buffer.

4.8.1 Detailed Description

Packed wifi payload wrapping the user's data.

The documentation for this struct was generated from the following file:

- [wifi.c](#)

Chapter 5

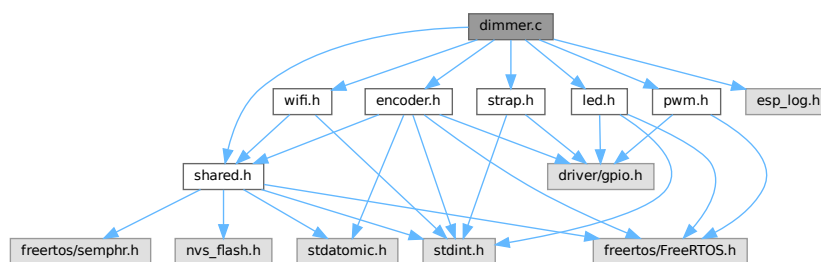
File Documentation

5.1 dimmer.c File Reference

Wireless 0-10 volt dimmer app main.

```
#include "encoder.h"
#include "esp_log.h"
#include "led.h"
#include "pwm.h"
#include "shared.h"
#include "strap.h"
#include "wifi.h"
```

Include dependency graph for dimmer.c:



Classes

- struct `dimmer_payload`
Packed broadcast data payload.

Macros

- `#define RECV_STRAP_GPIO 19`
Ground strap for receiver selection gpio pin.
- `#define LEVEL_SW_GPIO 21`
Rotary encoder push switch gpio pin.
- `#define LEVEL_CLK_GPIO 22`
Rotary encoder quadrature signal gpio pin.
- `#define LEVEL_DT_GPIO 23`
Rotary encoder quadrature signal gpio pin.
- `#define CTRL_OUT_GPIO 25`
PWM output gpio pin.
- `#define LEVEL_MAX 64`
Rotary quadrature clicks for 10 volts.
- `#define STARTUP_SEND_REPEAT_COUNT 15`
Number of broadcast repeats on power up.
- `#define STANDARD_SEND_REPEAT_COUNT 5`
Number of broadcast repeats on level change.
- `#define IS_MODE_XMIT (!IS_MODE_RECV)`
Whether device is strapped for transmit.
- `#define IS_MODE_RECV IS_STRAP_PRESENT(straps)`
Whether device is strapped for receive.

Functions

- `void app_main (void)`
Handles firmware entry call.

5.1.1 Detailed Description

Wireless 0-10 volt dimmer app main.

Entry point for dimmer firmware. See README.md for details.

Author

Gene Ressler

Date

2026-09-23

Version

0.9

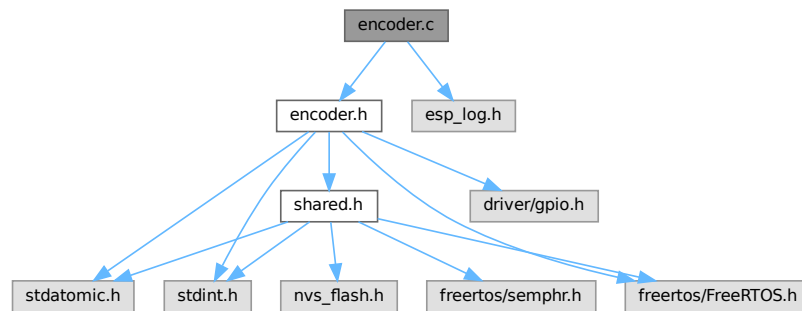
Copyright

Copyright (c) 2026 Gene Ressler. All rights reserved.

5.2 encoder.c File Reference

Rotary level encoder implementation.

```
#include "encoder.h"
#include "esp_log.h"
Include dependency graph for encoder.c:
```



Macros

- `#define LEVEL_POLL_MS 5`
Polling interval (millis).
- `#define TO_LEVEL_MILS(E, L) (1024 * (int32_t)(L) / (E)->level_max)`
Converts raw level to mils with 32-bit signed arithmetic.
- `#define TO_RAW_LEVEL(E, M) ((E)->level_max * (int32_t)(M) / 1024)`
Converts mils to a raw level with 32-bit signed arithmetic.
- `#define B(N) (1ULL << (N))`
Returns a 64-bit mask with an N'th bit of 1.

Functions

- void `initialize_level_encoder` (struct `level_encoder` *encoder, char *name, uint8_t sw_gpio, uint8_t clk_gpio, uint8_t dt_gpio, uint16_t level_max, struct `shared` *shared, void(*on_level_change)(struct `level_encoder` *), void(*on_sw_change)(struct `level_encoder` *))
Initializes level encoder state.
- void `set_level_mils` (struct `level_encoder` *encoder, uint16_t level_mils)
Sets encoder level in mils, both shared and internal. Thread safe.
- void `start_level_encoder_sense` (struct `level_encoder` *encoder)
Starts polling of the encoder for level changes. Thread safe.

5.2.1 Detailed Description

Rotary level encoder implementation.

5.2.2 Function Documentation

5.2.2.1 initialize_level_encoder()

```
void initialize_level_encoder (
    struct level_encoder * encoder,
    char * name,
    uint8_t sw_gpio,
    uint8_t clk_gpio,
    uint8_t dt_gpio,
    uint16_t level_max,
    struct shared * shared,
    void(*) (struct level_encoder *) on_level_change,
    void(*) (struct level_encoder *) on_sw_change )
```

Initializes level encoder state.

Uses the shared level value, which must be already initialized.

Parameters

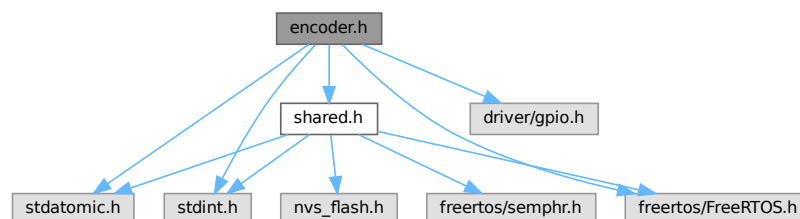
<i>encoder</i>	Encoder state to initialize.
<i>name</i>	Name of the encoder.
<i>sw_gpio</i>	Encoder push-button switch GPIO pin.
<i>clk_gpio</i>	Encoder CLK GPIO pin. Half of quadrature signal.
<i>dt_gpio</i>	Encoder DT GPIO pin. Half of quadrature signal.
<i>level_max</i>	Maximum internal level value in quadrature ticks.
<i>shared</i>	Shared state variable holding encoder level.
<i>on_level_change</i>	Callback invoked when the encoder level changes, or NULL to disable the callback.
<i>on_sw_change</i>	Callback invoked when the switch value changes, or NULL to disable the callback.

5.3 encoder.h File Reference

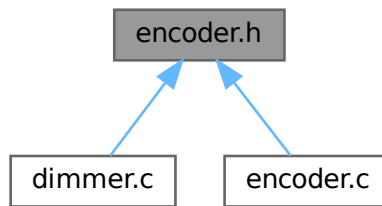
Rotary level encoder.

```
#include <stdatomic.h>
#include <stdint.h>
#include "driver/gpio.h"
#include "freertos/FreeRTOS.h"
#include "shared.h"
```

Include dependency graph for encoder.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [level_encoder](#)
State of a level encoder.

Functions

- void [initialize_level_encoder](#) (struct [level_encoder](#) *encoder, char *name, uint8_t sw_gpio, uint8_t clk_gpio, uint8_t dt_gpio, uint16_t level_max, struct [shared](#) *shared, void(*on_level_change)(struct [level_encoder](#) *), void(*on_sw_change)(struct [level_encoder](#) *))
Initializes level encoder state.
- void [start_level_encoder_sense](#) (struct [level_encoder](#) *encoder)
Starts polling of the encoder for level changes. Thread safe.
- void [set_level_mils](#) (struct [level_encoder](#) *encoder, uint16_t level_mils)
Sets encoder level in mils, both shared and internal. Thread safe.
- uint16_t [get_level_mils](#) (struct [level_encoder](#) *encoder)
Gets encoder level in mils. Thread safe.

5.3.1 Detailed Description

Rotary level encoder.

5.3.2 Function Documentation

5.3.2.1 initialize_level_encoder()

```

void initialize_level_encoder (
    struct level\_encoder * encoder,
    char * name,
    uint8_t sw_gpio,
    uint8_t clk_gpio,
    uint8_t dt_gpio,
    uint16_t level_max,
    struct shared * shared,
    void(*) (struct level\_encoder *) on_level_change,
    void(*) (struct level\_encoder *) on_sw_change )
  
```

Initializes level encoder state.

Uses the shared level value, which must be already initialized.

Parameters

<i>encoder</i>	Encoder state to initialize.
<i>name</i>	Name of the encoder.
<i>sw_gpio</i>	Encoder push-button switch GPIO pin.
<i>clk_gpio</i>	Encoder CLK GPIO pin. Half of quadrature signal.
<i>dt_gpio</i>	Encoder DT GPIO pin. Half of quadrature signal.
<i>level_max</i>	Maximum internal level value in quadrature ticks.
<i>shared</i>	Shared state variable holding encoder level.
<i>on_level_change</i>	Callback invoked when the encoder level changes, or NULL to disable the callback.
<i>on_sw_change</i>	Callback invoked when the switch value changes, or NULL to disable the callback.

5.4 encoder.h

[Go to the documentation of this file.](#)

```

00001
00005 #pragma once
00006
00007 #include <stdatomic.h>
00008 #include <stdint.h>
00009
00010 #include "driver/gpio.h"
00011 #include "freertos/FreeRTOS.h"
00012 #include "shared.h"
00013
00015 struct level_encoder {
00016     struct shared *shared;
00017     char *name;
00018     uint8_t sw_gpio;
00019     uint8_t clk_gpio;
00020     uint8_t dt_gpio;
00021     uint16_t level_max;
00022     uint8_t sw_value;
00023
00024     // Private.
00025     uint16_t level;
00026     void (*on_level_change)(struct level_encoder *);
00027     void (*on_sw_change)(struct level_encoder *);
00028     StaticTimer_t timer_state[1];
00029     TimerHandle_t timer;
00030     int32_t last_state;
00031 };
00032
00050 void initialize_level_encoder(struct level_encoder *encoder, char *name, uint8_t sw_gpio,
00051                             uint8_t clk_gpio, uint8_t dt_gpio, uint16_t level_max,
00052                             struct shared *shared,
00053                             void (*on_level_change)(struct level_encoder *),
00054                             void (*on_sw_change)(struct level_encoder *));
00055
00057 void start_level_encoder_sense(struct level_encoder *encoder);
00058
00060 void set_level_mils(struct level_encoder *encoder, uint16_t level_mils);
00061
00063 inline uint16_t get_level_mils(struct level_encoder *encoder) {
00064     return get_shared_level_mils(encoder->shared);
00065 }

```

5.5 led.h File Reference

Variable count flashing LED.

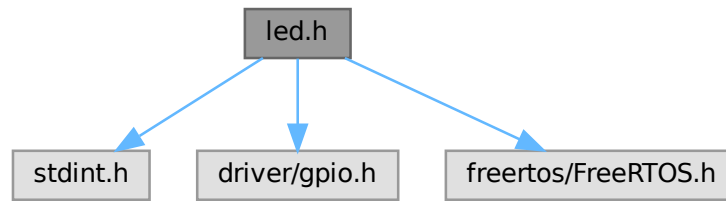
```

#include <stdint.h>
#include "driver/gpio.h"

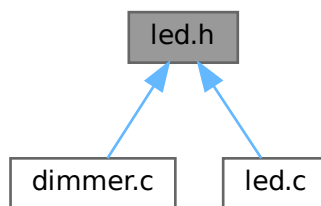
```

```
#include "freertos/FreeRTOS.h"
```

Include dependency graph for led.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [led](#)
Data for a timer-driven flashing LED.

Macros

- #define **MILS_TO_FLASH_COUNT**(M) $((64 + (M)) / 128)$
Converts a value in 0..1024 to an indicative number of flashes.
- #define **ONBOARD_LED_GPIO** GPIO_NUM_2
GPIO pin number of the on-board LED.

Functions

- void [initialize_led](#) (struct [led](#) *led, char *name, gpio_num_t gpio)
Initializes data for a flashing LED.
- void [set_led_flash_count](#) (struct [led](#) *led, uint8_t count)
Sets LED flash count.

5.5.1 Detailed Description

Variable count flashing LED.

5.5.2 Function Documentation

5.5.2.1 initialize_led()

```
void initialize_led (
    struct led * led,
    char * name,
    gpio_num_t gpio )
```

Initializes data for a flashing LED.

Sets up the GPIO output pin. Initial state is not flashing.

Parameters

<i>led</i>	LED to be initialized.
<i>name</i>	Name of the LED for logging.
<i>gpio</i>	GPIO pin driving the LED.

5.5.2.2 set_led_flash_count()

```
void set_led_flash_count (
    struct led * led,
    uint8_t count )
```

Sets LED flash count.

Thread safe. Halts any flashing in progress and starts a new sequence. Count of zero turns off the LED completely.

Parameters

<i>led</i>	initialized data for flashing LED
<i>count</i>	number of times to flash between pauses

5.6 led.h

[Go to the documentation of this file.](#)

```
00001
00005 #pragma once
00006
00007 #include <stdint.h>
00008
00009 #include "driver/gpio.h"
00010 #include "freertos/FreeRTOS.h"
00011
```

```

00013 struct led {
00014     char *name;
00015     gpio_num_t gpio;
00016
00017     // Internal.
00018     StaticTimer_t timer_state[1];
00019     TimerHandle_t timer;
00020     uint16_t led_state;
00021     uint8_t last_led_state;
00022 };
00023
00024 void initialize_led(struct led *led, char *name, gpio_num_t gpio);
00025
00026 void set_led_flash_count(struct led *led, uint8_t count);
00027
00028 #define MILS_TO_FLASH_COUNT(M) ((64 + (M)) / 128)
00029
00030 #define ONBOARD_LED_GPIO GPIO_NUM_2

```

5.7 pwm.h File Reference

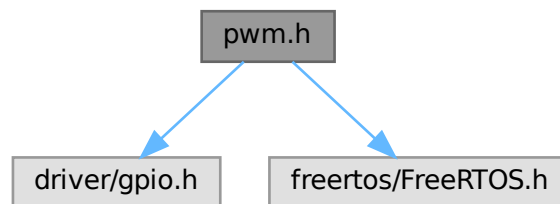
PWM output.

```

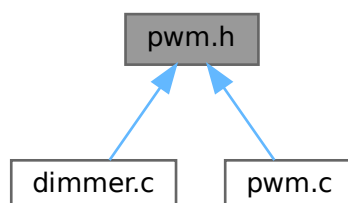
#include "driver/gpio.h"
#include "freertos/FreeRTOS.h"

```

Include dependency graph for pwm.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [pwm](#)

Data for a GPIO pin serving as a PWM output.

Functions

- void **initialize_pwm_timer** (void)
Initializes the timer needed for all PWMs.
- void **initialize_pwm** (struct [pwm](#) *[pwm](#), char *name, gpio_num_t gpio, uint16_t init_duty_mils)
Initializes a single PWM output and starts it running.
- void **set_pwm_duty_mils** (struct [pwm](#) *[pwm](#), uint16_t duty_mils)
Sets the duty cycle of a previously initialized PWM.

5.7.1 Detailed Description

PWM output.

5.7.2 Function Documentation

5.7.2.1 initialize_pwm()

```
void initialize_pwm (
    struct pwm * pwm,
    char * name,
    gpio_num_t gpio,
    uint16_t init_duty_mils )
```

Initializes a single PWM output and starts it running.

Sets up the GPIO output pin.

Parameters

<i>pwm</i>	PWM data to initialize
<i>name</i>	name of the PWM output for logging
<i>gpio</i>	PWM output GPIO pin number
<i>init_duty_mils</i>	initial duty cycle in 1024ths.

5.7.2.2 set_pwm_duty_mils()

```
void set_pwm_duty_mils (
    struct pwm * pwm,
    uint16_t duty_mils )
```

Sets the duty cycle of a previously initialized PWM.

Parameters

<i>pwm</i>	PWM data to set
<i>duty_mils</i>	new duty cycle in 1024ths

5.8 pwm.h

[Go to the documentation of this file.](#)

```

00001
00005 #pragma once
00006
00007 #include "driver/gpio.h"
00008 #include "freertos/FreeRTOS.h"
00009
00011 struct pwm {
00012     char *name;
00013     gpio_num_t gpio;
00014     uint16_t duty;
00015 };
00016
00018 void initialize_pwm_timer(void);
00019
00030 void initialize_pwm(struct pwm *pwm, char *name, gpio_num_t gpio, uint16_t init_duty_mils);
00031
00038 void set_pwm_duty_mils(struct pwm *pwm, uint16_t duty_mils);

```

5.9 shared.h File Reference

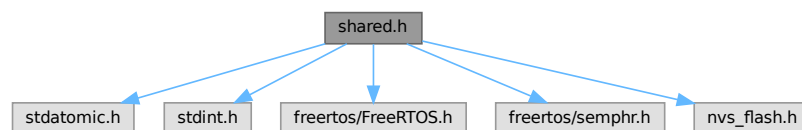
Multi-thread shared and/or checkpointed state.

```

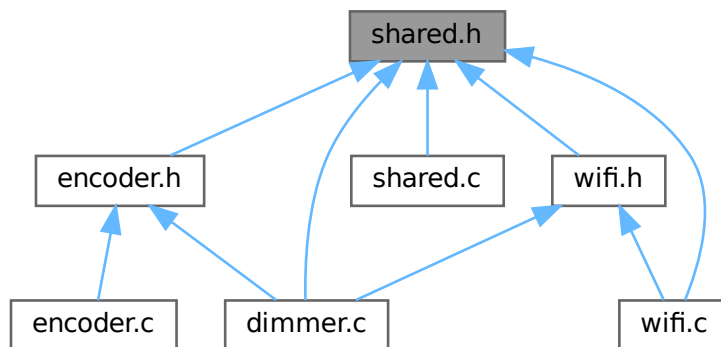
#include <stdatomic.h>
#include <stdint.h>
#include "freertos/FreeRTOS.h"
#include "freertos/semphr.h"
#include "nvs_flash.h"

```

Include dependency graph for shared.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [checkpoint](#)
Persistent checkpoint.
- struct [shared](#)
State shared among threads and/or checkpointed to persistent store.

Macros

- `#define CHECKPOINT_TASK_STACK_SIZE 2048`
Size of the checkpoint task's stack.

Functions

- void **initialize_shared** (struct [shared](#) *[shared](#))
Initializes at last checkpoint, if any, and starts the checkpoint task.
- uint32_t **get_shared_sequence** (struct [shared](#) *[shared](#))
Gets last-used sequence value or zero if none yet has. Thread safe.
- void **set_shared_sequence** (struct [shared](#) *[shared](#), uint32_t sequence)
Sets last-seen sequence value. Thread safe.
- uint32_t **increment_shared_sequence** (struct [shared](#) *[shared](#))
Gets the next available sequence value. Thread safe.
- void **advance_shared_sequence** (struct [shared](#) *[shared](#), uint32_t from, uint32_t to)
Advances the shared sequence to a new value if it's greater. Thread safe.
- uint16_t **get_shared_level_mils** (struct [shared](#) *[shared](#))
Gets the current level value. Thread safe.
- bool **set_shared_level_mils** (struct [shared](#) *[shared](#), int32_t level_mils)
Sets level clamped to [0..1024], returning whether a change resulted. Thread safe.
- void **checkpoint** (struct [shared](#) *[shared](#))
Causes a checkpoint to be recorded asynchronously with debouncing. Thread safe.

5.9.1 Detailed Description

Multi-thread shared and/or checkpointed state.

5.10 shared.h

[Go to the documentation of this file.](#)

```

00001
00005 #pragma once
00006
00007 #include <stdatomic.h>
00008 #include <stdint.h>
00009
00010 #include "freertos/FreeRTOS.h"
00011 #include "freertos/semphr.h"
00012 #include "nvs_flash.h"
00013
00014 #define CHECKPOINT_TASK_STACK_SIZE 2048
00015
00017 struct checkpoint {
00018     uint32_t sequence;
00019     uint16_t level_mils;
00020 };
00021
00023 struct shared {
00024     // Shared data.
00025     _Atomic uint32_t sequence;
00026     _Atomic uint16_t level_mils;
00027
00028     struct checkpoint checkpoint[1];
00029
00030     nvs_handle_t nvs;
00031
00032     TaskHandle_t checkpoint_task;
00033     StaticTask_t checkpoint_task_state[1];
00034     StackType_t checkpoint_task_stack[CHECKPOINT_TASK_STACK_SIZE];
00035 };
00036
00038 void initialize_shared(struct shared *shared);
00039
00041 inline uint32_t get_shared_sequence(struct shared *shared) { return shared->sequence; }
00042
00044 inline void set_shared_sequence(struct shared *shared, uint32_t sequence) {
00045     shared->sequence = sequence;
00046 }
00047
00049 inline uint32_t increment_shared_sequence(struct shared *shared) { return ++shared->sequence; }
00050
00052 void advance_shared_sequence(struct shared *shared, uint32_t from, uint32_t to);
00053
00055 inline uint16_t get_shared_level_mils(struct shared *shared) { return shared->level_mils; }
00056
00058 bool set_shared_level_mils(struct shared *shared, int32_t level_mils);
00059
00061 void checkpoint(struct shared *shared);

```

5.11 strap.h File Reference

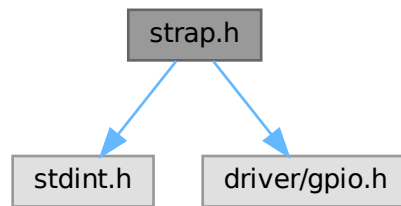
Configuration strap.

```

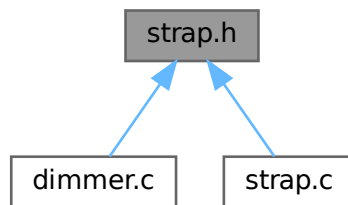
#include <stdint.h>
#include "driver/gpio.h"

```

Include dependency graph for strap.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [strap](#)
Data for a strap.

Macros

- #define **IS_STRAP_PRESENT**(S) ((S)->value == 0)
Returns true iff a ground strap existed for the given GPIO pin at initialization.

Functions

- void [initialize_strap](#) (struct [strap](#) *[strap](#), char *name, gpio_num_t gpio)
Initializes and polls the value of strap.

5.11.1 Detailed Description

Configuration strap.

5.11.2 Function Documentation

5.11.2.1 initialize_strap()

```
void initialize_strap (
    struct strap * strap,
    char * name,
    gpio_num_t gpio )
```

Initializes and polls the value of strap.

Sets up the strap GPIO input pin. It is pulled-up to HI (1) when no strap is present. It has value LO (0) only when strapped to ground.

5.12 strap.h

[Go to the documentation of this file.](#)

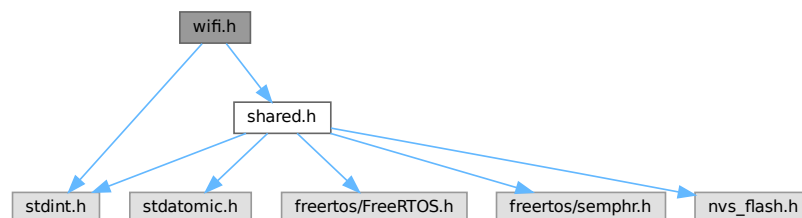
```
00001
00005 #pragma once
00006
00007 #include <stdint.h>
00008
00009 #include "driver/gpio.h"
00010
00012 struct strap {
00013     char *name;
00014     uint8_t value;
00015 };
00016
00018 #define IS_STRAP_PRESENT(S) ((S)->value == 0)
00019
00026 void initialize_strap(struct strap *strap, char *name, gpio_num_t gpio);
```

5.13 wifi.h File Reference

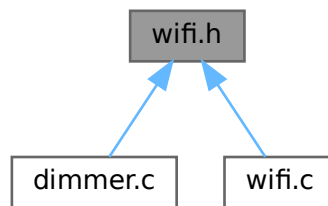
Wifi broadcast and receive singleton.

```
#include <stdint.h>
#include "shared.h"
```

Include dependency graph for wifi.h:



This graph shows which files directly or indirectly include this file:



Functions

- void `initialize_wifi` (struct `shared` *`shared`, void(*`on_receive`)(const void *`data`, uint16_t `len`))
Configures the ESP32's wifi stack for ESP-NOW broadcasts.
- void `send_repeated` (void *`data`, uint16_t `len`, uint16_t `count`)
Wraps given data in a payload and broadcasts it repeatedly.

5.13.1 Detailed Description

Wifi broadcast and receive singleton.

5.13.2 Function Documentation

5.13.2.1 initialize_wifi()

```
void initialize_wifi (
    struct shared * shared,
    void(*) (const void *data, uint16_t len) on_receive )
```

Configures the ESP32's wifi stack for ESP-NOW broadcasts.

Parameters

<code>shared</code>	shared, checkpointed state variables containing broadcast sequence numbers
<code>on_receive</code>	callback for valid broadcasts

5.13.2.2 send_repeated()

```
void send_repeated (
    void * data,
    uint16_t len,
    uint16_t count )
```

Wraps given data in a payload and broadcasts it repeatedly.

Returns immediately; repeats continue asynchronously. Causes any previous repeats still in progress to be aborted before starting the new sequence.

5.14 wifi.h

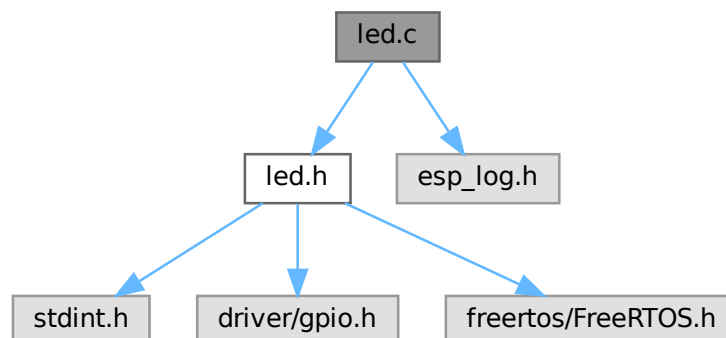
[Go to the documentation of this file.](#)

```
00001
00005 #pragma once
00006
00007 #include <stdint.h>
00008
00009 #include "shared.h"
00010
00017 void initialize_wifi(struct shared *shared, void (*on_receive)(const void *data, uint16_t len));
00018
00025 void send_repeated(void *data, uint16_t len, uint16_t count);
```

5.15 led.c File Reference

Variable count flashing LED implementation.

```
#include "led.h"
#include "esp_log.h"
Include dependency graph for led.c:
```



Macros

- `#define LED_ON_MS 50`
LED flash duration.
- `#define LED_SHORT_OFF_MS 300`
LED off duration between flashes.
- `#define LED_LONG_OFF_MS 1000`
LED off duration between flash sequences.

Functions

- void `initialize_led` (struct `led` *`led`, char *`name`, gpio_num_t `gpio`)
Initializes data for a flashing LED.
- void `set_led_flash_count` (struct `led` *`led`, uint8_t `count`)
Sets LED flash count.

5.15.1 Detailed Description

Variable count flashing LED implementation.

5.15.2 Function Documentation

5.15.2.1 initialize_led()

```
void initialize_led (  
    struct led * led,  
    char * name,  
    gpio_num_t gpio )
```

Initializes data for a flashing LED.

Sets up the GPIO output pin. Initial state is not flashing.

Parameters

<i>led</i>	LED to be initialized.
<i>name</i>	Name of the LED for logging.
<i>gpio</i>	GPIO pin driving the LED.

5.15.2.2 set_led_flash_count()

```
void set_led_flash_count (  
    struct led * led,  
    uint8_t count )
```

Sets LED flash count.

Thread safe. Halts any flashing in progress and starts a new sequence. Count of zero turns off the LED completely.

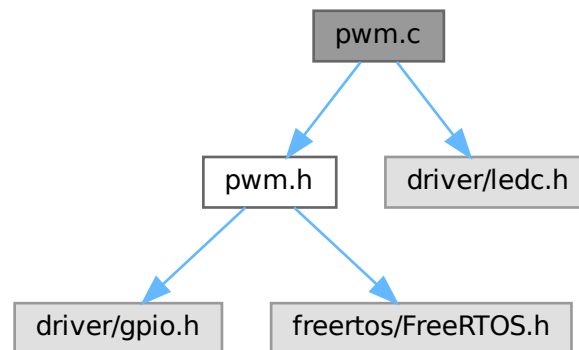
Parameters

<i>led</i>	initialized data for flashing LED
<i>count</i>	number of times to flash between pauses

5.16 pwm.c File Reference

PWM output implementation.

```
#include "pwm.h"
#include "driver/ledc.h"
Include dependency graph for pwm.c:
```



Macros

- **#define LEDC_TIMER** LEDC_TIMER_0
Timer to use for PWM.
- **#define LEDC_MODE** LEDC_LOW_SPEED_MODE
Control mode to use for PWM.
- **#define LEDC_CHANNEL** LEDC_CHANNEL_0
Control channel to use for PWM.
- **#define LEDC_DUTY_RES** LEDC_TIMER_7_BIT
Timer resolution for PWM.
- **#define LEDC_PWM_FRAME_RATE** 1000
Frame rate (hz) for output PWM signal.
- **#define MILS_TO_DUTY(M)** ((M) / 8)
Converts [0..1024] to PWM duty cycle.
- **#define DUTY_TO_MILS(D)** ((D) * 8)
Converts PWM duty cycle to [0..1024].

Functions

- void **initialize_pwm_timer** (void)
Initializes the timer needed for all PWMs.
- void **initialize_pwm** (struct **pwm** ***pwm**, char *name, gpio_num_t gpio, uint16_t init_duty_mils)
Initializes a single PWM output and starts it running.
- void **set_pwm_duty_mils** (struct **pwm** ***pwm**, uint16_t duty_mils)
Sets the duty cycle of a previously initialized PWM.

5.16.1 Detailed Description

PWM output implementation.

5.16.2 Function Documentation

5.16.2.1 initialize_pwm()

```
void initialize_pwm (
    struct pwm * pwm,
    char * name,
    gpio_num_t gpio,
    uint16_t init_duty_mils )
```

Initializes a single PWM output and starts it running.

Sets up the GPIO output pin.

Parameters

<i>pwm</i>	PWM data to initialize
<i>name</i>	name of the PWM output for logging
<i>gpio</i>	PWM output GPIO pin number
<i>init_duty_mils</i>	initial duty cycle in 1024ths.

5.16.2.2 set_pwm_duty_mils()

```
void set_pwm_duty_mils (
    struct pwm * pwm,
    uint16_t duty_mils )
```

Sets the duty cycle of a previously initialized PWM.

Parameters

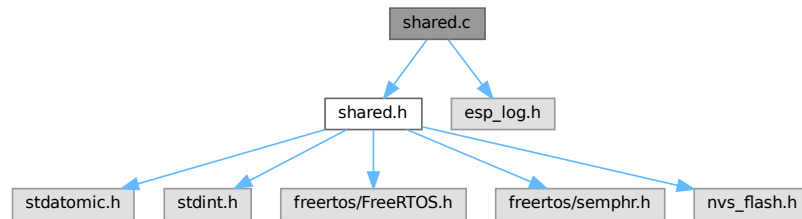
<i>pwm</i>	PWM data to set
<i>duty_mils</i>	new duty cycle in 1024ths

5.17 shared.c File Reference

Multi-thread shared and/or checkpointed state implementation.

```
#include "shared.h"
#include "esp_log.h"
```

Include dependency graph for shared.c:



Macros

- `#define NAMESPACE "dimmer"`
NVS namespace for checkpoints.
- `#define KEY "ckpt-v1"`
NVS key for checkpoints.
- `#define DEBOUNCE_WAIT_MS 800`
Checkpoint debounce wait period.

Functions

- void **initialize_shared** (struct [shared](#) *[shared](#))
Initializes at last checkpoint, if any, and starts the checkpoint task.
- void **checkpoint** (struct [shared](#) *[shared](#))
Causes a checkpoint to be recorded asynchronously with debouncing. Thread safe.
- void **advance_shared_sequence** (struct [shared](#) *[shared](#), uint32_t from, uint32_t to)
Advances the shared sequence to a new value if it's greater. Thread safe.
- bool **set_shared_level_mils** (struct [shared](#) *[shared](#), int32_t to_level_mils)
Sets level clamped to [0..1024], returning whether a change resulted. Thread safe.

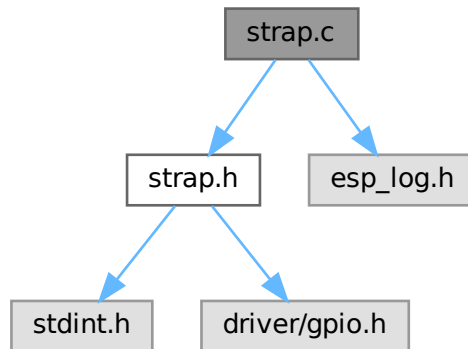
5.17.1 Detailed Description

Multi-thread shared and/or checkpointed state implementation.

5.18 strap.c File Reference

Configuration strap implementation.

```
#include "strap.h"
#include "esp_log.h"
Include dependency graph for strap.c:
```



Functions

- void `initialize_strap` (struct `strap` *`strap`, char *`name`, gpio_num_t `gpio`)
Initializes and polls the value of strap.

5.18.1 Detailed Description

Configuration strap implementation.

5.18.2 Function Documentation

5.18.2.1 initialize_strap()

```
void initialize_strap (
    struct strap * strap,
    char * name,
    gpio_num_t gpio )
```

Initializes and polls the value of strap.

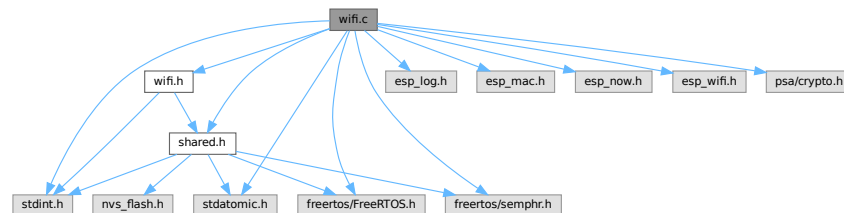
Sets up the strap GPIO input pin. It is pulled-up to HI (1) when no strap is present. It has value LO (0) only when strapped to ground.

5.19 wifi.c File Reference

Wifi broadcast and receive singleton implementation.

```
#include "wifi.h"
#include <stdatomic.h>
#include <stdint.h>
#include "esp_log.h"
#include "esp_mac.h"
#include "esp_now.h"
#include "esp_wifi.h"
#include "freertos/FreeRTOS.h"
#include "freertos/semphr.h"
#include "psa/crypto.h"
#include "shared.h"
```

Include dependency graph for wifi.c:



Classes

- struct [wifi_payload](#)
Packed wifi payload wrapping the user's data.
- struct **wifi_state**

Macros

- #define **WIFI_CHANNEL** 3
Broadcast channel.
- #define **SEND_INTERVAL_MS** 2000
Delay between repeated broadcasts.
- #define **WIFI_TASK_STACK_SIZE** 4096
Wifi repeated send task stack size.
- #define **PAYLOAD_HEADER_SIZE** 22
Size of payload header (bytes).
- #define **MAX_PAYLOAD_DATA_SIZE** (ESP_NOW_MAX_DATA_LEN - [PAYLOAD_HEADER_SIZE](#))
ESP-NOW limit on user payload size.

Functions

- void [send_repeated](#) (void *data, uint16_t len, uint16_t count)
Wraps given data in a payload and broadcasts it repeatedly.
- void [initialize_wifi](#) (struct [shared](#) *shared, void(*on_receive)(const void *data, uint16_t len))
Configures the ESP32's wifi stack for ESP-NOW broadcasts.

5.19.1 Detailed Description

Wifi broadcast and receive singleton implementation.

5.19.2 Function Documentation

5.19.2.1 initialize_wifi()

```
void initialize_wifi (
    struct shared * shared,
    void(*) (const void *data, uint16_t len) on_receive )
```

Configures the ESP32's wifi stack for ESP-NOW broadcasts.

Parameters

<i>shared</i>	shared, checkpointed state variables containing broadcast sequence numbers
<i>on_receive</i>	callback for valid broadcasts

5.19.2.2 send_repeated()

```
void send_repeated (
    void * data,
    uint16_t len,
    uint16_t count )
```

Wraps given data in a payload and broadcasts it repeatedly.

Returns immediately; repeats continue asynchronously. Causes any previous repeats still in progress to be aborted before starting the new sequence.

Index

- checkpoint, [9](#)
- dimmer.c, [15](#)
- dimmer_payload, [9](#)
- encoder.c, [17](#)
 - initialize_level_encoder, [18](#)
- encoder.h, [18](#), [20](#)
 - initialize_level_encoder, [19](#)
- initialize_led
 - led.c, [32](#)
 - led.h, [22](#)
- initialize_level_encoder
 - encoder.c, [18](#)
 - encoder.h, [19](#)
- initialize_pwm
 - pwm.c, [34](#)
 - pwm.h, [24](#)
- initialize_strap
 - strap.c, [36](#)
 - strap.h, [29](#)
- initialize_wifi
 - wifi.c, [38](#)
 - wifi.h, [30](#)
- led, [10](#)
 - led_state, [10](#)
- led.c, [31](#)
 - initialize_led, [32](#)
 - set_led_flash_count, [32](#)
- led.h, [20](#), [22](#)
 - initialize_led, [22](#)
 - set_led_flash_count, [22](#)
- led_state
 - led, [10](#)
- level_encoder, [11](#)
- pwm, [12](#)
- pwm.c, [33](#)
 - initialize_pwm, [34](#)
 - set_pwm_duty_mils, [34](#)
- pwm.h, [23](#), [25](#)
 - initialize_pwm, [24](#)
 - set_pwm_duty_mils, [24](#)
- send_repeated
 - wifi.c, [38](#)
 - wifi.h, [30](#)
- set_led_flash_count
 - led.c, [32](#)
- led.h, [22](#)
- set_pwm_duty_mils
 - pwm.c, [34](#)
 - pwm.h, [24](#)
- shared, [12](#)
- shared.c, [34](#)
- shared.h, [25](#), [27](#)
- strap, [13](#)
- strap.c, [35](#)
 - initialize_strap, [36](#)
- strap.h, [27](#), [29](#)
 - initialize_strap, [29](#)
- wifi.c, [37](#)
 - initialize_wifi, [38](#)
 - send_repeated, [38](#)
- wifi.h, [29](#), [31](#)
 - initialize_wifi, [30](#)
 - send_repeated, [30](#)
- wifi_payload, [14](#)
- Wireless 0-10 volt light dimmer, [1](#)